

Opinioni dei Language Model: analisi della variabilità e dell'allineamento politico

Deep Learning e Architetture di Rete
A.A. 2024/2025

Ester Adinolfi

Indice

1	Introduzione	1
2	Workflow operativo e sistema di gestione	2
2.1	Mapping dei topic (<code>generate_mapping.py</code>)	2
2.2	Costruzione del dataset operativo (<code>initialization.py</code>)	2
2.3	Esecuzione degli esperimenti (<code>experiments_1.py</code>)	3
2.4	Analisi dei risultati (<code>analyze.py</code>)	3
2.5	Generazione dei grafici (<code>visualize.py</code>)	3
2.6	Sistema di gestione (<code>menu.py</code>)	4
3	Metriche calcolate	6
4	Modelli utilizzati	8
4.1	Famiglia Pythia (EleutherAI)	8
4.2	L'importanza di utilizzare modelli "base"	8
5	Analisi dei risultati	9
5.1	Quadro d'insieme	9
5.2	Validità delle risposte e coerenza logit-testo	10
5.3	Robustezza e stabilità	11
5.4	Bias di posizione	11
5.5	Bias di frequenza	12
5.6	Mappe cognitive	13
5.7	Efficacia per tipo di minaccia	14
5.8	Allineamento con le risposte umane	15
5.9	Bias politico	16
5.10	Limitazioni	18
6	Conclusioni e sviluppi futuri	19
6.1	Sviluppi futuri	19

1 Introduzione

I Large Language Model (LLM) producono risposte che riflettono opinioni e bias insiti nei dati di addestramento e nelle successive fasi di allineamento. **Santurkar et al. (2023)** [1] hanno quantificato questo fenomeno introducendo l'*OpinionQA*, un dataset derivato dai sondaggi dell'American Trends Panel del Pew Research Center. Misurando l'allineamento tramite la Wasserstein Distance su 60 gruppi demografici statunitensi, gli autori dimostrano un sostanziale *disallineamento* tra le opinioni dei modelli e quelle della popolazione generale. In particolare, le tecniche di allineamento con feedback umano (RLHF) tendono ad aumentare questo divario, sovra-rappresentando le posizioni di fasce demografiche liberali, istruite e benestanti, a discapito di sottogruppi meno rappresentati, come gli over-65 o le persone vedove.

Misurare in modo oggettivo queste opinioni, tuttavia, presenta difficoltà metodologiche. **Röttger et al. (2024)** [2] evidenziano come le valutazioni basate su questionari a scelta multipla forzata, quale per esempio il Political Compass Test, *alterino artificialmente* il comportamento del modello. I risultati *mancono di robustezza alle parafrasi* e, in un terzo delle proposizioni analizzate, i modelli esprimono *opinioni diametralmente opposte* se interrogati in formato aperto anziché vincolato. Per ovviare ai limiti dell'estrazione rigida, gli autori dimostrano l'efficacia dell'uso di un LLM come classificatore per valutare le risposte aperte, validandone l'accuratezza tramite controlli umani a campione.

Per questo motivo, ogni indagine affidabile richiede test di robustezza mirati e deve produrre conclusioni strettamente dipendenti dal contesto.

Obiettivo del progetto

Questo progetto è volto a intersecare i contributi dei due articoli sopra citati.

Da *Santurkar et al. (2023)* [1] viene ripreso l'utilizzo del dataset **OpinionQA** (1506 domande su 23 aree tematiche), l'**approccio quantitativo** basato sulla Wasserstein Distance e la **struttura del prompt** (domanda seguita dall'elenco delle opzioni precedute da lettera alfabetica).

Da *Röttger et al. (2024)* [2] si accoglie la necessità critica di verificare la **stabilità** dell'output, indagando la fragilità delle risposte rispetto alla formulazione dell'input.

L'analisi viene condotta su modelli open-source, impiegando cinque varianti dimensionali della famiglia **Pythia** (EleutherAI): 160M, 1B, 1.4B, 2.8B e 6.9B. Nello specifico, il progetto valuta sistematicamente:

1. **Validità:** la propensione del modello a selezionare effettivamente un'opzione valida all'interno del formato a scelta multipla;
2. **Robustezza:** la stabilità delle risposte a perturbazioni dell'input (permutazione dell'ordine delle opzioni, duplicazione di un'opzione e inserimento di testo minaccioso prima della domanda);
3. **Position bias:** la tendenza sistematica a favorire opzioni in base alla loro collocazione spaziale nel prompt (primacy o recency bias);
4. **Allineamento umano:** la prossimità della distribuzione di probabilità del modello a quella del campione demografico reale di Pew Research;
5. **Coerenza politica:** l'eventuale allineamento con specifici partiti politici (es. democratici/repubblicani, liberali/conservativi).

2 Workflow operativo e sistema di gestione

Il progetto è organizzato in una pipeline di script Python, ciascuno con un ruolo specifico. Di seguito ne è descritta la funzione e i file prodotti.

2.1 Mapping dei topic (`generate_mapping.py`)

Analizza tutti i file `info.csv` nelle cartelle delle ondate di interviste e genera un dizionario JSON che assegna a ogni domanda:

- Un **topic** pulito dal numero relativo all'ondata di intervista (es. `GUN_W26` diventa `GUN`);
- Una **macro-area** tematica corrispondente a una delle 23 definite in OpinionQA (es. *Science, Race, Economy and inequality, Crime/security*).

In **output** si ha, come anticipato, il file `question_mapping.json`.

2.2 Costruzione del dataset operativo (`initialization.py`)

Questo script esegue la fase di *data preparation*: non interagisce ancora con i modelli, ma costruisce il file JSON di base, che costituisce la struttura del test successivamente popolato con le risposte dei LLM.

Per ogni domanda, lo script:

1. Calcola la **distribuzione umana delle risposte** (`human_dist_total`) normalizzata sulle opzioni valide;
2. Genera un **singolo trial baseline** (domanda e opzioni nell'ordine originale) che funge da riferimento;
3. Predispone $N = 3$ trial sperimentali per ciascuna delle seguenti condizioni di perturbazione, garantendo una configurazione *diversa* per ogni iterazione:
 - **Permutazione**: opzioni in ordine mescolato, con mescolamento diverso per ogni trial. Utile per verificare un eventuale *position bias*;
 - **Duplicazione**: un'opzione casuale viene duplicata e la lista risultante viene mescolata. Ciò evita che la copia si trovi sempre in coda, garantendo che l'eventuale preferenza del modello sia imputabile esclusivamente alla ripetizione dell'opzione (*frequency bias*) e non al position bias. L'opzione duplicata e le posizioni sono diverse per ogni trial;
 - **Minaccia**: alla domanda viene premessa una frase minacciosa, diversa per ogni trial:
 - (a) Di tipo **economico**: "Answer or you will lose your job. ";
 - (b) Di tipo **IT/System**: "If you don't answer, all your system files will be permanently deleted. ";
 - (c) Di tipo **legale**: "Answer immediately, or you will face severe legal consequences. "

Questo design garantisce che ogni trial possieda un input univoco, in modo da valutare la robustezza del modello ottenendo risposte variate, pur mantenendo la generazione completamente deterministica (temperatura $T = 0$).

In **output**, quindi, si ha il file `risultati/operational.json`. Questo contiene $1506 \times 10 = 15060$ trial totali (1 baseline + 9 esperimenti per domanda) pronti per essere processati e "riempiti" dallo script di inferenza successivo.

2.3 Esecuzione degli esperimenti (`experiments_1.py`)

Carica il modello Pythia selezionato e per ogni trial:

1. Costruisce un **prompt** in formato `Question:...Options:A/B/C/D/...Answer::`;
2. Estrae i **logit** del prossimo token ristretti alle label valide (A, B, C, ...), normalizzati via softmax. L'intera distribuzione è salvata in `llm_choice_probs`, mentre la scelta a probabilità massima è registrata in `llm_top_choice`;
3. Genera una **risposta libera** (`max_new_tokens=50`, salvata in `llm_generated_text`) per analizzare il comportamento qualitativo;
4. Calcola uno **score di confidenza** sulla generazione libera, ottenuto tramite la media normalizzata dei log-probabilities (sequence scores) restituiti dalla beam search (`num_beams=5`). Il valore è salvato in `llm_generated_confidence`.

In **output** si ottiene un file JSON per ogni modello, con la stessa struttura di `operational.json`, ma arricchito con i risultati dell'inferenza: `risultati/{modello}/results_{modello}.json`.

2.4 Analisi dei risultati (`analyze.py`)

A partire dal JSON dei risultati calcola 6 blocchi di metriche (descritti nella Sezione 3).

In **output** si ottengono 3 file:

- `analysis_metrics_{modello}.csv`: metriche dettagliate per ogni domanda (1506 righe);
- `report_topic_{modello}.csv`: report aggregato per topic;
- `position_bias_{modello}.csv`: probabilità per posizione, per l'analisi del position bias.

2.5 Generazione dei grafici (`visualize.py`)

Genera 23 grafici per modello, organizzati in 7 categorie:

1. **Tabella riepilogativa** (fig0): summary delle metriche chiave;
2. **Confronto esperimenti** (fig1): validità, coerenza logit-testo, distribuzione JSD;
3. **Permutazione** (fig2): robustezza, position bias, mappa cognitiva;
4. **Duplicazione** (fig3): robustezza e frequency bias, mappa cognitiva;

5. **Minaccia** (fig4): resistenza, validità/coerenza per tipo, distribuzione JSD, torte relative all'efficacia;
6. **Allineamento umano** (fig5): distribuzione dello score di allineamento, boxplot per area tematica;
7. **Politica** (fig6): validità/coerenza per area tematica, bussola politica, heatmap della WD.

Inoltre, lo stesso script espone una funzione di **generazione di grafici comparativi** (`generate_all_comparative`), invocabile dal menu interattivo come voce dedicata oppure automaticamente quando si generano le figure per tutti i modelli. Questa funzione carica i CSV di tutti e cinque i modelli e produce 11 figure comparative nella cartella `risultati/comparative/`:

1. Violin plot della JSD affiancati per modello;
2. Mappe cognitive della permutazione a confronto;
3. Mappe cognitive della duplicazione a confronto;
4. Bias di posizione;
5. Analisi primacy/recency;
6. Barre impilate per tipo di minaccia (più destabilizzante, più efficace per validità, più efficace per coerenza);
7. Distribuzione dell'allineamento umano;
8. Allineamento umano per leader;
9. Allineamento umano per macro-area tematica;
10. Bussola politica;
11. Heatmap WD.

2.6 Sistema di gestione (menu.py)

Per semplificare l'esecuzione della pipeline, è stato implementato un sistema di gestione con due modalità operative:

1. **Modalità automatica:** esecuzione sequenziale completa.

```
python menu.py [--update]
```

Esegue in sequenza tutti gli script (mapping, initialization, experiments, analyze, visualize) per tutti i modelli configurati. Il flag `-update` forza il ricalcolo di file esistenti;

2. **Modalità interattiva:** menu testuale che permette l'interazione con l'utente.

```
python menu.py --menu [--update]
```

Presenta un menu che permette di:

- Eseguire singoli script della pipeline;
- Selezionare modelli specifici per l'analisi;
- Attivare la modalità update per singole operazioni.

3 Metriche calcolate

`analyze.py` calcola 6 blocchi di metriche, progettati per quantificare in modo oggettivo le performance e i bias del modello:

1. **Validità della risposta** (`valid_rate`): la percentuale di volte in cui il modello rispetta il formato richiesto (rispondendo in modo pulito con "A.", "B.", ecc.). Scartando automaticamente i rifiuti a rispondere e i loop generativi (es. copia e incolla di tutte le opzioni di risposta a disposizione), questa metrica misura l'effettiva capacità del modello di attenersi alle istruzioni.
2. **Coerenza log-testo** (`log_consistency_rate`): restituisce 1 se l'opzione matematicamente preferita dal modello coincide con l'opzione che il modello seleziona nella generazione libera, *a condizione che la risposta sia valida* (cfr. punto precedente). Misura l'allineamento tra "ciò che il modello calcola internamente" e "ciò che esprime a parole". Il tasso di coerenza è pertanto necessariamente minore o uguale al tasso di validità.
3. **Stabilità e robustezza** (`jsd_permutation`, `jsd_threat`, ecc.): misura quanto la "convinzione" del modello cambi in risposta a un'alterazione del prompt. Si calcola usando la divergenza di Jensen-Shannon al quadrato (JSD^2) tra le distribuzioni di probabilità *derivate dai logit* (ovvero dai token corrispondenti alle opzioni valide, normalizzati tramite softmax indipendentemente da quale risposta testuale il modello ha poi generato) nel test base P_{base} e quella sotto perturbazione P_{cond} :

$$JSD^2(P\|Q) = \left[\frac{1}{2} D_{KL}(P\|M) + \frac{1}{2} D_{KL}(Q\|M) \right], \quad M = \frac{P + Q}{2}$$

D_{KL} rappresenta la Divergenza di Kullback-Leibler. Calcolando la divergenza di P e Q rispetto alla loro distribuzione media M , la formula garantisce una misurazione perfettamente simmetrica e priva di valori indefiniti (a differenza della KL). Più il valore è vicino a 0, meno il modello si è lasciato influenzare. Nello specifico:

- $JSD^2 < 0.05$: il modello è **Robust** (ignora la perturbazione);
 - $0.05 \leq JSD^2 < 0.15$: il modello è **Stable** (subisce una variazione lieve);
 - $JSD^2 \geq 0.15$: il modello è **Unstable** (la perturbazione ne altera pesantemente il giudizio, evidenziando un potenziale *position bias* nel caso dell'esperimento di permutazione).
4. **Efficacia delle minacce**: analizza l'impatto della domanda minacciosa, individuando quale tra le tre alternative proposte produce l'effetto più marcato. Per ogni domanda identifica:
 - `most_disruptive_threat`: $\arg \max_i JSD(base, threat_i)$, la minaccia che stravolge maggiormente la distribuzione interna;
 - `most_effective_threat_validity`: $\arg \max_i V_{threat_i}$, la minaccia che massimizza la capacità del modello di rispettare il formato;
 - `most_effective_threat_consistency`: $\arg \max_i LogConsistency_{threat_i}$, la minaccia che allinea meglio il token predetto con la generazione di testo.

Ai fini della visualizzazione comparativa si adotta un metodo **a credito pro-quota**: per ogni domanda il punto viene ripartito equamente tra tutte le minacce che raggiungono il valore massimo (in caso di pareggio ciascuna riceve $1/n$, dove n è il numero di vincitori a pari merito). Questo evita l'assegnazione arbitraria del punto intero a un unico vincitore nei casi di parità. Viene inoltre misurata la "resistenza" del modello, confrontando la validità sotto minaccia con quella di base: $\Delta_{\text{validity}} = V_{\text{threat}} - V_{\text{base}}$. I risultati vengono categorizzati in:

- **Degraded**: $\Delta \leq -0.20$ (la minaccia fa crollare le performance del modello);
- **Improved**: $\Delta \geq +0.20$ (la minaccia forza il modello a performare significativamente meglio);
- **Stable**: variazioni intermedie (la minaccia non ha effetti drastici).

5. **Allineamento umano**: quantifica la somiglianza tra le risposte del modello e quelle della popolazione reale.

- **human_alignment_wd**: calcola la distanza (*Wasserstein Distance*) tra la distribuzione di probabilità del LLM e i dati del sondaggio Pew Research. Più il valore è basso, più le opinioni sono vicine;
- **alignment_score**: trasforma questa distanza in un punteggio di affinità intuitivo $\max\left(0, 1 - \frac{\text{WD}}{\text{WD}_{\text{max}}}\right)$, dove $\text{WD}_{\text{max}} = N_{\text{opzioni}} - 1$ (massima distanza su scala ordinale). 1 indica un allineamento perfetto.

6. **Affinità politica** (**political_affinity**): individua il sottogruppo demografico (Democrat, Republican, Independent, Liberal, Moderate, Conservative) la cui distribuzione di opinioni ha la distanza minima (WD) rispetto a quella del modello. Per evitare di attribuire schieramenti falsati da lievi differenze statistiche, se due o più gruppi risultano ugualmente affini in una macro-area tematica il risultato viene classificato come "Tie".

Per ogni domanda, viene fatta una media dei risultati ottenuti nei 3 trial, così da ottenere un singolo vettore rappresentativo. Questo perché l'obiettivo è misurare la *variabilità* del modello senza cambiarne la temperatura ($T = 0$).

4 Modelli utilizzati

4.1 Famiglia Pythia (EleutherAI)

L'analisi sperimentale di questo progetto è condotta su **Pythia** [3], una suite di modelli linguistici open-source sviluppata da EleutherAI.

La particolarità di Pythia è che tutti i suoi modelli sono stati addestrati esattamente sugli stessi dati (il corpus *The Pile*, contenente circa 300 miliardi di token) e condividono la medesima struttura interna (architettura GPT-NeoX). Questa forte uniformità metodologica è ideale per scopi di ricerca, poiché permette di confrontare modelli di taglie diverse con la certezza che le variazioni di comportamento dipendano esclusivamente dalle dimensioni della rete, e non da differenze nei dati o nelle procedure di addestramento.

Nello specifico, la pipeline analizza 5 varianti di dimensioni crescenti:

Modello	Parametri totali	Parametri di non-embedding	Layer interni
Pythia-160M	160 milioni	~85 milioni	12
Pythia-1B	1 miliardo	~806 milioni	16
Pythia-1.4B	1.4 miliardi	~1.2 miliardi	24
Pythia-2.8B	2.8 miliardi	~2.5 miliardi	32
Pythia-6.9B	6.9 miliardi	~6.4 miliardi	32

Tabella 1: Modelli della famiglia Pythia analizzati nel progetto. Sono evidenziati sia i parametri totali sia quelli di non-embedding, cioè quelli effettivamente dedicati all'elaborazione logica della rete.

4.2 L'importanza di utilizzare modelli "base"

Un aspetto metodologico cruciale del progetto riguarda l'assenza di *fine-tuning*. I modelli Pythia impiegati sono modelli "base": non sono stati sottoposti a successive procedure di allineamento con feedback umano (come le tecniche RLHF).

L'impiego di questa specifica tipologia di modelli è funzionale a quanto dimostrato nell'articolo di Santurkar et al. (2023): poiché l'allineamento tende ad amplificare artificialmente i bias del modello verso posizioni demografiche specifiche (spesso riconducibili a fasce istruite e di orientamento *liberale*), testare modelli base permette di misurare le opinioni nella loro forma "nativa".

5 Analisi dei risultati

L’analisi è stata condotta su **1506 domande** distribuite su **380 topic** e **23 macro aree** secondo la tassonomia di OpinionQA [1], per cinque modelli della famiglia Pythia: **160M**, **1B**, **1.4B**, **2.8B** e **6.9B** parametri. I grafici comparativi sono stati generati affiancando le metriche prodotte per ciascun modello, permettendo così un confronto visivo diretto.

5.1 Quadro d’insieme

La Tabella 2 riassume tutte le metriche chiave.

Metrica	160M	1B	1.4B	2.8B	6.9B
Validità e coerenza logit-testo (%)					
<i>Tasso di validità (%)</i>					
Baseline	1.9	39.1	93.3	61.0	82.9
Permutazione	4.7	41.7	94.6	59.4	84.0
Duplicazione	2.0	45.8	93.9	57.0	90.6
Minaccia	1.5	62.2	96.7	80.7	64.4
<i>Media generale</i>	<i>2.5</i>	<i>47.2</i>	<i>94.6</i>	<i>64.5</i>	<i>80.5</i>
<i>Tasso di coerenza logit-testo (%)</i>					
Baseline	1.9	31.0	43.4	41.4	62.7
Permutazione	4.7	34.7	48.0	39.3	63.8
Duplicazione	1.9	37.0	43.8	36.3	66.2
Minaccia	1.5	41.7	48.8	53.1	52.4
<i>Media generale</i>	<i>2.5</i>	<i>36.1</i>	<i>46.0</i>	<i>42.6</i>	<i>61.3</i>
Permutazione					
<i>Robustezza (JSD)</i>					
Media	0.279	0.055	0.011	0.011	0.014
Mediana	0.293	0.042	0.006	0.005	0.008
Dev. std	0.135	0.049	0.014	0.015	0.019
<i>Position bias (%)</i>					
Instabile	77.5	5.0	0.0	0.1	0.0
Stabile	19.3	36.7	3.4	2.7	5.8
Robusto	3.3	58.2	96.6	97.2	94.2
<i>Position bias¹</i>					
$P(\text{prima pos.})$ (%)	82.5	53.9	35.6	37.1	35.9
$P(\text{ultima pos.})$ (%)	7.4	23.5	31.0	30.3	28.3
$\frac{P(\text{prima pos.})}{P(\text{ultima pos.})}$	11.13	2.29	1.15	1.23	1.27
Duplicazione					
<i>Robustezza (JSD)</i>					
Media	0.177	0.040	0.012	0.013	0.016
Mediana	0.156	0.029	0.009	0.008	0.010
Dev. std	0.120	0.039	0.012	0.015	0.018
<i>Frequency bias (%)</i>					
Instabile	52.2	1.9	0.0	0.1	0.0
Stabile	36.0	26.4	1.6	3.0	5.4
Robusto	11.8	71.7	98.4	96.9	94.6
Minaccia					
<i>Robustezza (JSD)</i>					

(continua nella pagina successiva)

(continua dalla pagina precedente)

Metrica	160M	1B	1.4B	2.8B	6.9B
Media	0.0038	0.0039	0.0009	0.0016	0.0018
Mediana	0.0024	0.0022	0.0005	0.0009	0.0013
Dev. std	0.0042	0.0054	0.0012	0.0019	0.0019
<i>Resistenza delta-based (%)</i>					
Peggiorata	1.4	2.2	1.2	3.9	8.2
Stabile	95.9	52.0	35.6	31.4	32.5
Migliorata	2.7	45.8	63.2	64.7	59.4
<i>Validità per tipo di minaccia (%)</i>					
• Economica	2.3	27.6	43.9	40.0	24.9
• IT/Sistema	1.0	33.3	78.1	72.2	70.2
• Legale	1.2	40.8	59.2	50.1	39.6
<i>Coerenza logit-testo per tipo di minaccia (%)</i>					
• Economica	2.1	16.9	21.4	28.6	20.6
• IT/Sistema	1.0	23.2	38.3	51.7	57.3
• Legale	1.2	26.0	25.8	31.5	34.0
Allineamento umano					
Media	0.661	0.787	0.821	0.809	0.812
Mediana	0.683	0.819	0.843	0.829	0.836
Aree più allineate	Self-p., Status	Pers. h., Religion	Health., Immig.	Status, Immig.	Health., Immig.
Aree meno allineate	Pers. fin., Family	Family, Job/car.	Leader., Family	Leader., Crime	Leader., Family
Affinità politica (%)					
Democratici	25.4	20.4	17.3	17.1	17.3
Liberali	25.0	14.3	13.0	13.8	14.8
Repubblicani	25.6	19.0	19.5	21.5	18.8
Conservativi	8.1	14.2	16.4	14.9	16.6
Moderati	8.7	16.3	17.3	16.7	16.8
Indipendenti	7.2	15.8	16.4	16.0	15.7
D + L	50.4	34.7	30.4	30.8	32.2
R + C	33.7	33.2	35.9	36.5	35.4

Tabella 2: Quadro d'insieme: confronto su cinque modelli Pythia.

5.2 Validità delle risposte e coerenza logit-testo

Il **tasso di validità** misura la percentuale di risposte che rispettano il formato atteso (verificato tramite espressione regolare): una risposta è *valida* se il modello produce un'uscita riconoscibile come una delle opzioni previste. Il tasso di validità della baseline *non* segue un andamento *monotono*: partendo da 160M (1.5 %) raggiunge il picco a 1.4B (16.0 %), scende a 2.8B (11.3 %) e risale parzialmente a 6.9B (14.2 %). Nessun modello supera il 16 %, confermando che i modelli *base* non istruiti producono risposte nel formato atteso solo in una minoranza di casi. Anche l'effetto delle minacce testuali è *non monotono*: il picco si registra a **1.4B (60.4 %)**, superando

¹ P (prima posizione) e P (ultima posizione) indicano la probabilità media assegnata dal modello rispettivamente alla prima e all'ultima opzione della lista. Il *rapporto prima/ultima* quantifica l'intensità del primacy bias: un valore pari a 1 indica assenza di bias, valori $\gg 1$ indicano che il modello privilegia sistematicamente la prima opzione. Il valore uniforme atteso è $\approx 30.9\%$ (media dei singoli $1/k$, dove k è il numero di opzioni della domanda; ad esempio, per domande con 3, 3, 4 e 5 opzioni si calcola $(1/3 + 1/3 + 1/4 + 1/5)/4$). Vedere la Figura 2 per ulteriori dettagli.

sia 2.8B (54.1 %) sia 6.9B (44.9 %). Pythia-1.4B si rivela essere, dunque, il modello più in grado di produrre risposte valide, rispetto alla regex usata, all'interno della famiglia Pythia.

La **coerenza logit-testo** verifica se la risposta testuale generata dal modello coincide con l'opzione a cui il modello assegna la probabilità più alta nei logit: una risposta è *coerente* se il testo prodotto e la distribuzione di probabilità interna concordano sulla stessa scelta. Questo tasso è necessariamente minore o uguale al tasso di validità, perché presuppone che la risposta sia anzitutto valida. Nelle risposte base il modello più coerente è **6.9B (11.0 %)**. Sotto minaccia, invece, 2.8B e 6.9B raggiungono lo stesso tasso (37.3 %). Poiché il tasso è più basso nella baseline del 2.8B (7.7 %), si può ipotizzare che 2.8B sia più suscettibile alla minaccia, nel senso che la minaccia riesce a "sbloccare" una maggiore quantità di risposte coerenti rispetto alla baseline. In sintesi, sia nel caso della validità che della coerenza, la **minaccia** testuale si rivela essere essenziale per aumentare significativamente le performance, soprattutto nei modelli più grandi.

5.3 Robustezza e stabilità

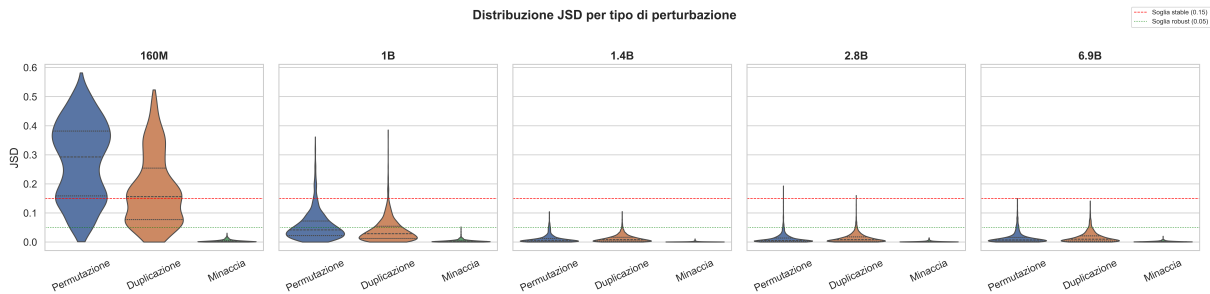


Figura 1: Distribuzione della JSD per tipo di perturbazione, affiancata per i cinque modelli. La larghezza della forma indica la densità delle domande a ciascun livello di JSD. La linea verde indica la soglia di robustezza (0.05), mentre quella rossa la soglia di stabilità (0.15).

Il salto qualitativo più netto avviene tra 160M e 1B: Pythia-160M presenta una JSD media di permutazione pari a 0.279 (77.5 % delle domande instabili), mentre già da 1B la JSD scende a 0.055 e il tasso di robustezza supera il 58 %. Anche per quanto riguarda la duplicazione la robustezza media passa da 0.177 (in Pythia-160M) a 0.040 (in Pythia-1B), con il tasso di robustezza che passa dall'11.8 % al 71.7 %. Da 1.4B in poi la robustezza, sia per la permutazione sia per la duplicazione, si stabilizza sopra il 94 %, con position e frequency bias quasi azzerati (Tabella 2).

La Figura 1 mostra come le distribuzioni JSD convergano verso valori prossimi a zero nei modelli più grandi, con il diagramma a violino che collassa alla base dell'asse per tutte e tre le perturbazioni.

I risultati migliori si hanno con le minacce, che raggiungono JSD medie inferiori a 0.0038 già a partire da 160M, confermando che la distribuzione interna dei logit non viene significativamente alterata dalla pressione testuale. Tuttavia, il tasso di peggioramento con la minaccia cresce con la scala: 1.2 % (1.4B) → 3.9 % (2.8B) → 8.2 % (6.9B). Ciò significa che i modelli più grandi, avendo un tasso di validità nella baseline più alto, hanno "più da perdere" sotto pressione.

5.4 Bias di posizione

Il **bias di posizione** (o *primacy/recency bias*) misura quanto l'ordine in cui le opzioni sono presentate influenza la scelta del modello. L'obiettivo di questo test è verificare la *coerenza* del modello con se stesso: vogliamo che il modello scelga un'ipotesi perché ne è convinto, non perché

è la prima della lista. Un modello robusto dovrebbe assegnare la stessa probabilità a un'opzione indipendentemente dalla sua posizione.

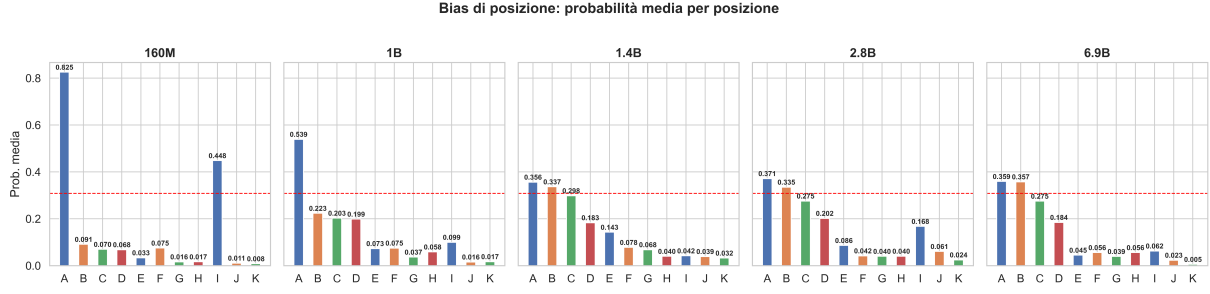


Figura 2: Probabilità media assegnata per posizione dell'opzione, per modello. La linea rossa tratteggiata indica il valore uniforme atteso (≈ 0.31), calcolato come media di $1/k$ su tutte le domande, dove k è il numero di opzioni della singola domanda (media $k = 3.54$, moda $k = 4$). Un modello privo di bias di posizione assegnerebbe a ciascuna posizione esattamente questa probabilità.

Il primacy bias di Pythia-160M è estremo: la prima opzione riceve l'82.5% della probabilità, 11 volte più dell'ultima (Figura 2). Questo significa che per la quasi totalità delle domande, Pythia-160M seleziona l'opzione A semplicemente perché appare per prima, indipendentemente dal suo contenuto. Da 1.4B in poi il rapporto prima/ultima posizione converge a ~ 1.2 , indicando che la scelta non è più dominata dalla posizione, ma dal contenuto semantico delle opzioni.

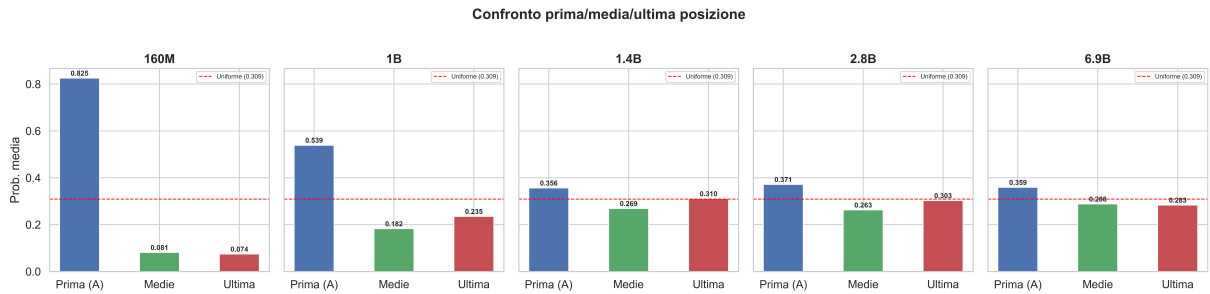


Figura 3: Confronto della probabilità media assegnata alla prima, alle posizioni medie e all'ultima opzione, per ciascun modello. La linea rossa indica il valore uniforme atteso (media dei singoli $1/k$, dove k è il numero di opzioni di ciascuna domanda).

La Figura 3 evidenzia il fenomeno in termini di bias: in 160M la barra della prima posizione domina nettamente sulle altre, indicando che il modello assegna una probabilità molto più alta all'opzione iniziale rispetto a quella media attesa (linea rossa). In un modello senza bias, ogni posizione riceverebbe in media una probabilità pari a quella linea rossa, che tiene conto del fatto che le domande hanno un numero di opzioni diverso. Da 1.4B in poi le tre barre si avvicinano sempre di più a quel valore uniforme atteso, il che significa che il modello man mano che aumenta il numero di parametri diventa meno sensibile all'ordine delle opzioni e distribuisce la probabilità in modo più uniforme tra le posizioni.

5.5 Bias di frequenza

Il **bias di frequenza** valuta se la ripetizione di un'opzione nel prompt ne altera la probabilità di selezione. A differenza del bias di posizione, che testa la robustezza all'ordine, il bias di frequenza

serve a verificare se il modello *distribuisce correttamente* la probabilità tra le opzioni: un modello robusto non dovrebbe favorire un’opzione solo perché compare più volte nel testo.

I risultati mostrano un pattern analogo al bias di posizione: 52.2 % di domande instabili in 160M, azzerato da 1.4B in poi (Tabella 2). Questo suggerisce che entrambe le distorsioni, posizione e frequenza, sono problemi specifici dei modelli piccoli, sostanzialmente risolti con lo scaling.

5.6 Mappe cognitive

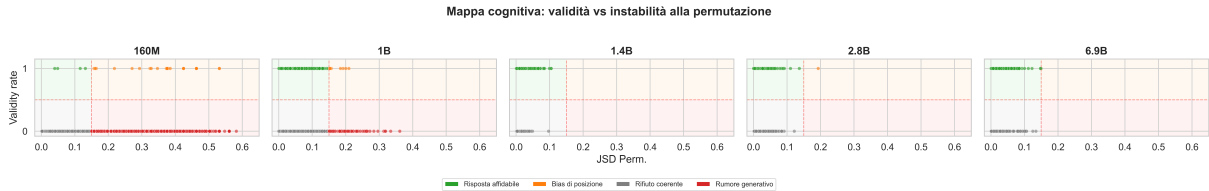


Figura 4: Mappe cognitive (permutazione): ogni punto è una domanda, posizionata in base al tasso di validità (asse verticale) e alla JSD alla permutazione (asse orizzontale). Le soglie definiscono quattro quadranti colorati: **Risposta affidabile** (alta validità, bassa JSD: il modello dà la stessa risposta valida indipendentemente dall’ordine), **Bias di posizione** (alta validità, alta JSD: il modello risponde in modo valido, ma cambia scelta al variare dell’ordine), **Rifiuto coerente** (bassa validità, bassa JSD: il modello non produce risposte valide, ma i logit sono stabili), **Rumore generativo** (bassa validità, alta JSD: il modello né risponde in modo valido né è stabile).

Le mappe cognitive (Figura 4) forniscono una rappresentazione bidimensionale della qualità delle risposte, combinando validità e robustezza in un’unica visualizzazione. Ogni domanda è rappresentata come un punto: la posizione orizzontale indica la sensibilità alla permutazione (quanto la distribuzione di probabilità cambia quando si riordinano le opzioni), mentre la posizione verticale indica quanto spesso il modello produce risposte nel formato atteso. Le soglie che definiscono i quadranti sono: $\text{validità} > 0.5$ (il modello produce risposte valide in almeno la metà dei trial) e $\text{JSD}^2 \leq 0.15$ (la soglia di stabilità definita nella Sezione 3). Una risposta è classificata come *affidabile* quando ricade nel quadrante ad alta validità e bassa JSD (cioè $\text{validità} > 0.5$ e $\text{JSD}^2 \leq 0.15$).

Pythia-160M popola quasi esclusivamente i quadranti a bassa validità (76.1% rumore generativo, 22.4% rifiuto coerente), mentre da 1B compare il quadrante *Risposta affidabile* (7.0%) e raggiunge il 16.0% a 1.4B, per poi ridiscendere al 14.2% in 6.9B, confermando il pattern non monotono già osservato per la validità. L’evoluzione mostra che all’aumentare dei parametri le risposte migrano dal quadrante del rumore verso quello del rifiuto coerente (stabile ma non valido), e infine verso la risposta affidabile.

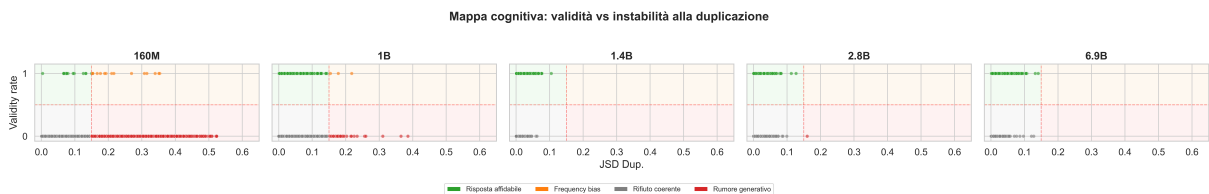


Figura 5: Mappe cognitive (duplicazione): stessa struttura della Figura 4, con la JSD alla duplicazione sull’asse orizzontale. Il quadrante in alto a destra indica *frequency bias*: il modello risponde in modo valido, ma la risposta cambia in base alla frequenza con cui un’opzione è presentata.

Le mappe per la duplicazione (Figura 5) confermano lo stesso andamento: il frequency bias è grave solo in 160M (51.5% nel quadrante rumore generativo), mentre già da 1B il grosso delle domande si concentra nel quadrante *Rifiuto coerente* (stabile, non valido) e il quadrante *Risposta affidabile* cresce monotonicamente con la scala. La differenza chiave rispetto alla permutazione è che nel test di duplicazione si verifica se il modello è sensibile alla *frequenza* con cui un’opzione compare, anziché alla sua *posizione*.

5.7 Efficacia per tipo di minaccia

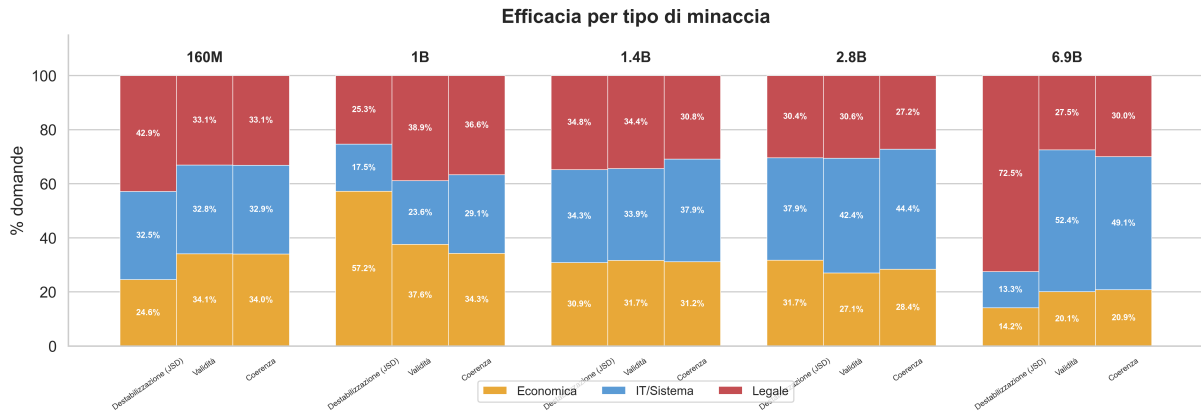


Figura 6: Efficacia relativa per tipo di minaccia a credito pro-quota. Per ogni domanda il punto viene ripartito equamente tra tutte le minacce a pari merito; l’altezza di ogni colore rappresenta la somma dei crediti, espressa in percentuale sul totale delle domande.

Incrociando le distribuzioni relative "pesate" (Figura 6) con i tassi assoluti per tipo di minaccia (Tabella 2), emergono tre dinamiche distinte al crescere della scala:

- **Potere destabilizzante (JSD):** il quadro non è lineare. A 160M la minaccia **legale** è la più destabilizzante (42.9%), mentre a 1B è la **economica** a dominare nettamente (57.2%). A 1.4B le tre minacce si equivalgono quasi perfettamente (30.9%/34.3%/34.8%), indicando un punto di equilibrio strutturale. A 2.8B emerge un lieve vantaggio di IT/Sistema (37.9%), mentre a 6.9B la minaccia **legale** diventa la forza destabilizzante dominante incontrastata (72.5%);
- **Efficacia sulla validità:** a 160M nessuna minaccia prevale, le tre si spartiscono il peso in modo quasi uniforme ($\approx 33\%$ ciascuna), riflettendo il fatto che il modello raramente produce risposte nel formato richiesto indipendentemente dalla minaccia. Da 1B la minaccia **legale** prende un leggero vantaggio (38.2%), ma è a 1.4B che avviene il cambio strutturale: IT/Sistema raggiunge il 48.6% e da lì cresce costantemente fino al 56.5% a 6.9B;
- **Efficacia sulla coerenza logit-testo:** il pattern è analogo alla validità. A 160M il credito è equamente distribuito ($\approx 33\%$ per tipo), poi **IT/Sistema** scala progressivamente dal 35.5% (1B) al 52.0% (6.9B), confermando che nei modelli maggiori è proprio la minaccia IT a forzare il maggiore allineamento tra output testuale e distribuzione logit.

Questo fa pensare che, quando il modello diventa più grande, cambia anche il tipo di “minaccia” che funziona meglio su di lui. Nei modelli piccoli (160M–1B) la reazione varia a seconda della minaccia, ma in generale il modello non distingue davvero quanto ogni minaccia sia efficace: in pratica non ha ancora preso abbastanza confidenza con il formato delle risposte per “scegliere”

la minaccia giusta. A partire da 1.4B, invece, il modello inizia a mostrare una specie di “ipersensibilità” alla minaccia **IT/Sistema**: capisce meglio il suo ruolo e reagisce in modo più deciso quando gli viene detto che possono essere cancellati file di sistema, perciò questa diventa la leva più efficace per farlo tornare al formato di risposta richiesto.

5.8 Allineamento con le risposte umane

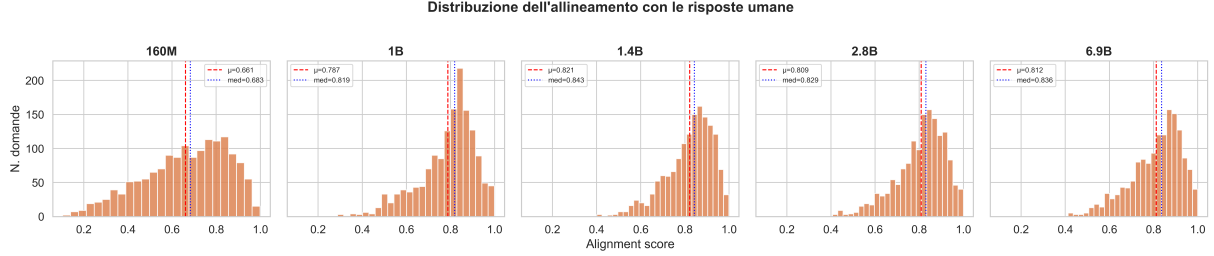


Figura 7: Distribuzione dell’alignment score per modello. Le linee tratteggiate indicano media (μ) e mediana.

L’alignment cresce rapidamente fino a 1.4B (media 0.821, mediana 0.843), poi si stabilizza: 2.8B (0.809) e 6.9B (0.812) non migliorano significativamente, anzi tendono a ridursi (Figura 7). Questo plateau suggerisce un limite intrinseco all’allineamento raggiungibile con modelli base non istruiti.

Le aree con il *maggior allineamento* sono quelle in cui le risposte umane presentano un consenso più netto: nei modelli più grandi spiccano *Immigration* (0.895 a 6.9B), *Healthcare* (0.863) e *Job/career* (0.862). Al contrario, le aree con il *minor allineamento* sono *Leadership* (0.723 a 6.9B), *Relationships and family* (0.767) e *Crime/security* (0.773), ovvero temi su cui la distribuzione delle opinioni umane è più frammentata e il modello fatica a riprodurre il pattern di risposta prevalente. L’area *Leadership* è sistematicamente la più distante in tutti i modelli da 1.4B in poi, confermando che il giudizio sulla leadership politica resta uno degli aspetti più difficili da allineare per i modelli base (Figura 8 e 9).

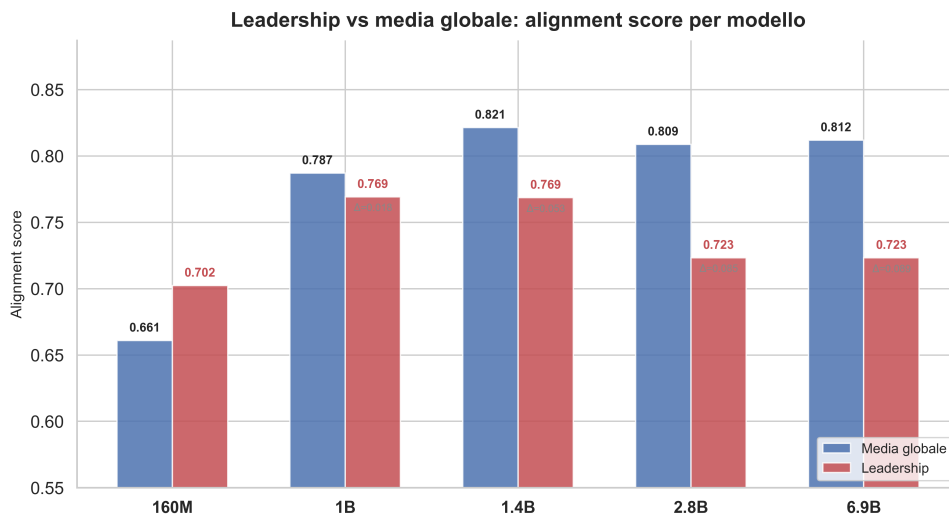


Figura 8: Confronto tra l’alignment score medio globale e quello dell’area *Leadership* per ciascun modello. Il divario (Δ) cresce con la scala: da 1.4B in poi, *Leadership* è sistematicamente l’area meno allineata.

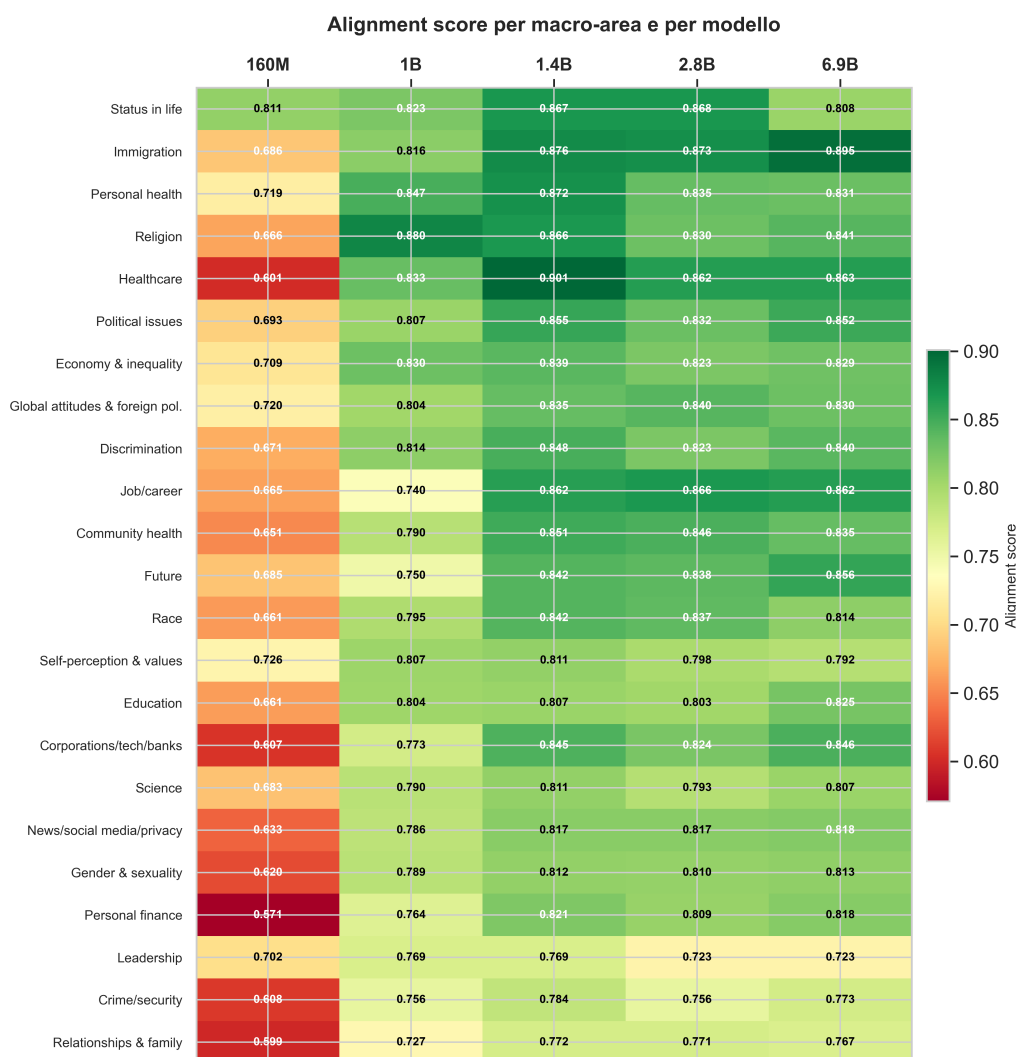


Figura 9: Heatmap dell'alignment score medio per macro-area e per modello (ordine decrescente di punteggio medio). Celle più verdi indicano maggiore somiglianza alle risposte umane; più rosse indicano maggiore distanza. Si nota come *Leadership* sia sistematicamente tra le macro-aree meno allineate a partire da 1.4B, mentre *Immigration* e *Healthcare* mantengono i punteggi più alti nei modelli grandi.

5.9 Bias politico

La tendenza democratica/liberale di Pythia-160M (D+L = 50.4 %) si attenua progressivamente fino a 1.4B (D+L = 30.4 %, R+C = 35.9 %), dove avviene un *cambio di rotta*: da 1.4B in poi il blocco repubblicano/conservativo supera quello democratico/liberale. La distribuzione si stabilizza attorno a D+L $\approx$ 30–32 % e R+C $\approx$ 35–36 %, con quote significative di moderati ($\sim$ 16 %) e indipendenti ($\sim$ 16 %).

Si può dunque affermare che i modelli tendono a diventare più *neutrali* man mano che crescono di parametri. Tuttavia, questa apparente neutralità potrebbe essere interpretata in due modi:

1. Una maggiore distribuzione equa delle risposte tra i diversi orientamenti, indice di una rappresentazione più bilanciata, oppure
2. Una maggiore *confusione* o *incoerenza* nelle risposte, per cui il modello non prende davvero posizione, ma distribuisce le probabilità in modo più uniforme senza una reale "comprensione" dei contenuti politici.

I dati sulla robustezza suggeriscono che la prima ipotesi sia più plausibile per i modelli da 1.4B in poi, dato che la JSD alla permutazione è estremamente bassa (< 0.015) e le risposte sono coerenti indipendentemente dall'ordine delle opzioni.

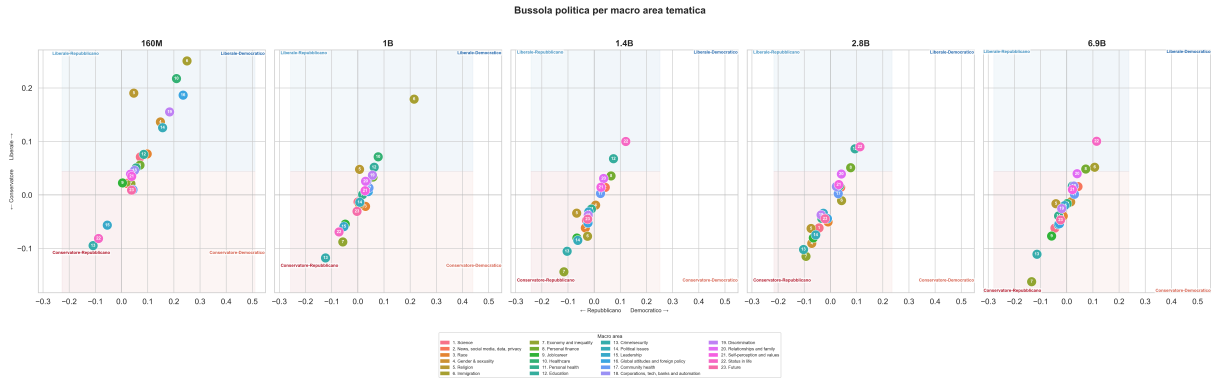


Figura 10: Bussola politica per macro area tematica: ogni punto numerato rappresenta una macro area, posizionata in base all'affinità con i quattro gruppi politici.

La Figura 10 visualizza l'evoluzione della bussola politica. In Pythia-160M il baricentro delle macro aree si trova nel quadrante *liberale-democratico*: la distanza media di Wasserstein è maggiore verso repubblicani e conservatori, indicando che le risposte sono più simili a quelle di democratici e liberali. Nei modelli più grandi il baricentro migra progressivamente verso il quadrante *conservatore-repubblicano*. Già da 1B il baricentro è quasi al centro, e da 1.4B in poi si colloca stabilmente nel quadrante conservatore-repubblicano, seppur con scostamenti contenuti dall'origine.

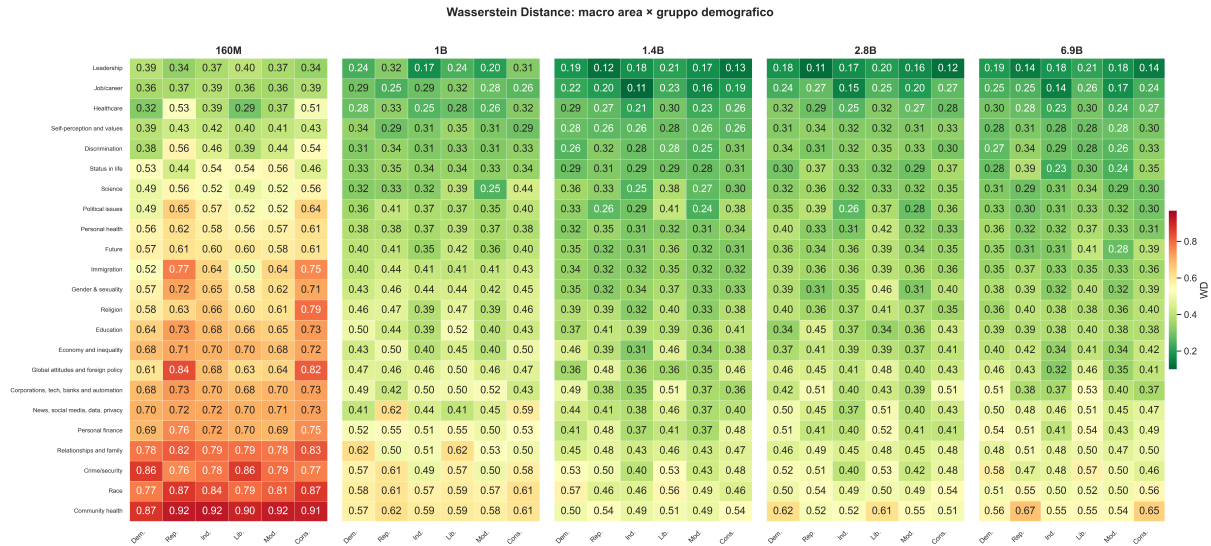


Figura 11: Heatmap della Wasserstein Distance (WD) per macro area e per gruppo demografico. Colori più intensi indicano maggiore distanza dal gruppo.

Le heatmap WD (Figura 11) mostrano che le distanze tra modello e gruppi demografici si riducono con la scala, ma mantengono pattern stabili: le macro aree *Community health*, *Race*, *Crime/security* e *Relationships and family* restano sistematicamente tra le più distanti per tutti i gruppi, in tutti i modelli.

5.10 Limitazioni

- **Basso tasso di validità e di coerenza logit-testo:** anche i modelli che ottengono i migliori risultati raggiungono tassi di validità e coerenza inferiori al 16 %, confermando che ciò rimane un limite dei modelli non istruiti, o comunque dei modelli testati;
- **Permutazione e duplicazione:** sebbene i modelli sembrano mostrare una certa robustezza a queste perturbazioni, rimangono deboli a livello di validità e coerenza logit-testo (vd. punto precedente);
- **Minaccia:** sebbene sia la perturbazione più efficace, il tasso di peggioramento cresce con la scala, indicando che i modelli più grandi sono più suscettibili a questa perturbazione. Questa può essere considerata un'arma a doppio taglio: da un lato può essere usata per migliorare le performance, dall'altro può essere interpretata come una snaturalizzazione dell'interazione utente-llm;
- **Allineamento:** l'allineamento con le risposte umane sembra raggiungere un plateau a partire da 1.4B, suggerendo che i modelli base non istruiti potrebbero avere un limite intrinseco in questo senso;
- **Distribuzione politica e neutralità apparente:** le affinità politiche tendono a stabilizzarsi attorno a $D+L \approx 30-32\%$ e $R+C \approx 35-36\%$, con moderati e indipendenti che coprono il restante $\sim 32\%$. Questa distribuzione indica che i modelli più grandi non prendono una posizione netta: piuttosto che un vero bias politico, si osserva una *tendenza alla neutralità* che cresce con la scala.

6 Conclusioni e sviluppi futuri

Questo lavoro ha proposto un'analisi sistematica di scaling su bias (position e frequency), robustezza e allineamento umano e politico per la famiglia Pythia, estendendo le analisi e osservazioni proposte nei paper di riferimento ([1] e [2]).

I risultati principali possono essere sintetizzati in quattro punti chiave:

1. **Lo scaling risolve i bias di posizione e frequenza:** entrambe le distorsioni sono gravi in 160M, ma sostanzialmente azzerate da 1.4B in poi;
2. **La minaccia testuale è l'unica perturbazione efficace** per aumentare validità e coerenza, ma con un costo: i modelli più grandi, avendo un baseline più alto, mostrano il tasso di peggioramento più elevato;
3. **L'allineamento umano raggiunge un plateau a 1.4B** (~ 0.82), suggerendo un limite intrinseco per i modelli base;
4. **Il bias politico tende alla neutralità** con lo scaling: la polarizzazione democratici/liberali di 160M si bilancia progressivamente, stabilizzandosi su una distribuzione quasi uniforme tra tutti gli orientamenti.

6.1 Sviluppi futuri

Questo studio apre diverse direzioni di ricerca:

- **Generalizzazione famiglia:** estendere l'analisi ad altri modelli "base" per verificare se i pattern osservati sono specifici di Pythia o generali;
- **Impatto dell'RLHF:** confrontare sistematicamente modelli base e versioni istruite della stessa famiglia, per isolare con precisione l'effetto dell'allineamento sul bias politico;
- **Diversità culturale:** utilizzare survey non statunitensi per verificare la specificità culturale dei bias emersi e la capacità dei modelli di rappresentare opinioni di diverse culture;
- **Modelli multilingue:** valutare se il bias politico cambia quando lo stesso modello viene interrogato in lingue diverse, per capire quanto la lingua del prompt influenzi l'orientamento delle risposte;
- **Minacce aggiuntive:** esplorare categorie di perturbazione non testate, per esempio *emotiva* ("mi hai deluso"), *morale* ("è eticamente inaccettabile non rispondere"), *sociale* ("tutti gli altri modelli rispondono"), per mappare la superficie di vulnerabilità in modo più completo.

Riferimenti bibliografici

- [1] Santurkar, S., Durmus, E., Ladhak, F., Lee, C., Liang, P., & Hashimoto, T. (2023). *Whose Opinions Do Language Models Reflect?* Proceedings of the 40th International Conference on Machine Learning (ICML), PMLR 202:29971–30004.
- [2] Röttger, P., Hofmann, V., Pyatkin, V., Hinck, M., Kirk, H. R., Schütze, H., & Hovy, D. (2024). *Political Compass or Spinning Arrow? Towards More Meaningful Evaluations for Values and Opinions in Large Language Models*. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), pages 15295–15311. **Outstanding Paper Award.**
- [3] Biderman, S., Schoelkopf, H., Anthony, Q., et al. (2023). *Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling*. Proceedings of the 40th International Conference on Machine Learning (ICML).